

VERIFYING THE SUM OF FOUR CUBES PROBLEM UP TO 10^9 VIA GENERALIZED PELL EQUATIONS

KAIYI ZHANG, ZHUO HUANG, ANYU WANG, AND XIAOYUN WANG

ABSTRACT. We computationally verify the sum of four cubes problem for every positive integer $n \leq 10^9$, producing explicit integer representations $n = x_1^3 + x_2^3 + x_3^3 + x_4^3$. The search is based on a transformation of a structured four-cube identity into generalized Pell equations $u^2 - dv^2 = M$. We also give a heuristic model, based on the square tests forced by primes dividing d and on conditional Bernays-type density, which explains the observed scale of the search parameter. The same method also quickly finds representations for large integers, such as $n = 9 \cdot 2^{128} + 5$, within seconds.

1. INTRODUCTION

1.1. Problem background. The sum of four cubes problem asks whether every integer n can be expressed as the sum of four integer cubes, i.e., whether there exist integers x_1, x_2, x_3, x_4 such that

$$(1.1) \quad x_1^3 + x_2^3 + x_3^3 + x_4^3 = n.$$

Unlike Waring’s problem for cubes, the variables in (1.1) are allowed to be negative.

1.2. Related work.

1.2.1. Waring’s problem. Waring’s problem concerns representing positive integers as sums of nonnegative k -th powers. For cubes, it is known that $g(3) = 9$ [6], meaning every positive integer can be represented as the sum of nine nonnegative cubes. This differs substantially from (1.1), where negative cubes are permitted.

1.2.2. Sum of three cubes. The sum of three cubes problem asks for integers x, y, z such that $x^3 + y^3 + z^3 = n$. Its congruence obstruction modulo 9 leaves only the classes $n \not\equiv \pm 4 \pmod{9}$ as possible, and even within those classes the search can be very hard. Booker resolved the notable case 33 [3], and Booker and Sutherland resolved 42 [4].

1.2.3. Sum of five cubes. The sum of five cubes problem is much easier. The identity [6]

$$(1.2) \quad 6x = (x+1)^3 + (x-1)^3 - x^3 - x^3,$$

shows that any multiple of 6 is a sum of four integer cubes. Since $r^3 \pmod{6}$ covers all residue classes, one additional cube is enough to represent any integer.

2020 *Mathematics Subject Classification.* Primary 11Y50; Secondary 11D25, 11P05, 11Y16.

Key words and phrases. Sum of four cubes, generalized Pell equations, computational verification, Diophantine equations, computational number theory.

Corresponding author: Anyu Wang.

1.2.4. *Sum of four cubes.* In 1966, Demjanenko [5] proved that every integer $n \not\equiv 4, 5 \pmod{9}$ is a sum of four integer cubes. The remaining classes $n \equiv \pm 4 \pmod{9}$ are the natural target for computation. Demjanenko's identities are constructive, but in some cases they produce large representations.

1.3. **Our contribution.** The present paper gives a direct computational verification for the full range $1 \leq n \leq 10^9$, including the classes $n \equiv \pm 4 \pmod{9}$. We do not claim that transforming four-cube identities into Pell-type equations is new in itself; related substitutions are part of the computational folklore around sums of cubes. The contribution here is instead threefold. First, we isolate a simple one-parameter family with explicit integrality and parity conditions. Second, we use this family to produce certificates for every $1 \leq n \leq 10^9$. Third, we provide an independently checkable verification protocol and a heuristic model for the observed search behavior.

2. PRELIMINARIES

2.1. **Generalized Pell equations.** A generalized Pell equation is a Diophantine equation of the form $x^2 - dy^2 = m$, where d is a non-square positive integer and m is an integer. When $m = 1$, it is called the standard Pell equation.

There are mature algorithms for solving generalized Pell equations (e.g., PQa algorithm [8], LMM algorithm [7]). These algorithms have wide applications in computational number theory and have efficient implementations. For example, the `solve_integer` method in SageMath [11] is based on the PARI/GP library [10] and Simon's reduction algorithm [9].

2.2. Demjanenko's Theorem.

Theorem 2.1 (Demjanenko, 1966 [5]). *For any integer n satisfying $n \not\equiv 4, 5 \pmod{9}$, there exist integers x_1, x_2, x_3, x_4 such that $x_1^3 + x_2^3 + x_3^3 + x_4^3 = n$.*

3. MAIN RESULTS

3.1. **Verified range.** The computation described in Section 5 produced, for every integer n with $1 \leq n \leq 10^9$, an explicit representation

$$x_1^3 + x_2^3 + x_3^3 + x_4^3 = n.$$

Each recorded quadruple was checked using exact integer arithmetic, and the verification protocol is described in Section 5.1.

3.2. Transformation to Pell equation.

Theorem 3.1 (Pell transformation with parity conditions). *Let $n \in \mathbb{Z}$ and let d be a positive integer. Put*

$$M_{n,d} = \frac{4n - 1 + d^3}{3}.$$

If $M_{n,d} \in \mathbb{Z}$, then the map

$$(x, y) \mapsto (u, v) = (2x + 1, 2y + d)$$

is a bijection between integer solutions of

$$(3.1) \quad (x + 1)^3 - x^3 - (y + d)^3 + y^3 = n$$

and integer solutions of

$$(3.2) \quad u^2 - dv^2 = M_{n,d}$$

satisfying

$$u \equiv 1 \pmod{2}, \quad v \equiv d \pmod{2}.$$

Moreover, $M_{n,d} \in \mathbb{Z}$ if and only if $d \equiv 1 - n \pmod{3}$.

Proof. Expanding the left-hand side of equation (3.1):

$$\begin{aligned} & (x+1)^3 - x^3 - (y+d)^3 + y^3 \\ &= (x^3 + 3x^2 + 3x + 1) - x^3 - (y^3 + 3dy^2 + 3d^2y + d^3) + y^3 \\ &= 3x^2 + 3x + 1 - 3dy^2 - 3d^2y - d^3. \end{aligned}$$

Rearranging terms:

$$(3.3) \quad 3x^2 + 3x - 3dy^2 - 3d^2y = n - 1 + d^3.$$

Multiplying both sides by 4 and completing the squares:

$$\begin{aligned} & 12x^2 + 12x - 12dy^2 - 12d^2y = 4n - 4 + 4d^3 \\ & 3(4x^2 + 4x) - 3d(4y^2 + 4dy) = 4n - 4 + 4d^3 \\ & 3[(2x+1)^2 - 1] - 3d[(2y+d)^2 - d^2] = 4n - 4 + 4d^3 \\ & (2x+1)^2 - 1 - d(2y+d)^2 + d^3 = \frac{4n - 4 + 4d^3}{3} \\ & u^2 - dv^2 = \frac{4n - 1 + d^3}{3}. \end{aligned}$$

Thus any integer pair (x, y) gives a solution of (3.2) with u odd and $v \equiv d \pmod{2}$. Conversely, if (u, v) satisfies (3.2) and these two parity conditions, then

$$x = \frac{u-1}{2}, \quad y = \frac{v-d}{2}$$

are integers, and substituting them in the displayed calculation gives (3.1). Finally, $4n - 1 + d^3 \equiv n - 1 + d \pmod{3}$, so $M_{n,d}$ is integral exactly when $d \equiv 1 - n \pmod{3}$. $\square$

The original identity also gives the parity obstruction $n \equiv 1 + d \pmod{2}$. Combining this with the integrality condition shows that it is enough to search

$$d \equiv 1 - n \pmod{6}.$$

Lemma 3.2 (Local obstructions). *Let $p^\alpha \parallel d$. If*

$$u^2 - dv^2 = M_{n,d}$$

has an integer solution, then $M_{n,d}$ is a square modulo p^α . In particular, for odd primes $p \neq 3$,

$$M_{n,d} \equiv (4n - 1) \cdot 3^{-1} \pmod{p^\alpha},$$

so $(4n - 1) \cdot 3^{-1}$ must be a square modulo p^α .

Proof. Reducing the equation modulo p^α gives $u^2 \equiv M_{n,d} \pmod{p^\alpha}$, since p^α divides d . If $p \neq 3$, then 3 is invertible modulo p^α , and $d^3 \equiv 0 \pmod{p^\alpha}$, giving the stated congruence. $\square$

3.3. Algorithm Description.

Algorithm 1 SolveFourCubes**Input:** Integer n **Output:** Integers X_1, X_2, X_3, X_4 such that $X_1^3 + X_2^3 + X_3^3 + X_4^3 = n$, or $\emptyset$ if no solution found

```

1: if  $n \bmod 9 = 4$  then
2:    $(X_1, X_2, X_3, X_4) \leftarrow \text{SOLVEFOURCUBES}(-n)$ 
3:   return  $(-X_1, -X_2, -X_3, -X_4)$ 
4: end if
5: for positive integers  $d \equiv 1 - n \pmod{6}$  do
6:   if  $d$  is a perfect square then
7:     continue
8:   end if
9:    $M \leftarrow (4n - 1 + d^3)/3$ 
10:  if  $M$  fails the local tests of Lemma 3.2 then
11:    continue
12:  end if
13:  Let  $\mathcal{C}$  be the quadruples obtained from solutions of  $u^2 - dv^2 = M$  subject
    to  $u \equiv 1 \pmod{2}$  and  $v \equiv d \pmod{2}$ 
14:  if  $\mathcal{C} = \emptyset$  then
15:    continue
16:  end if
17:  return the element of  $\mathcal{C}$  minimizing  $H = \max_i |X_i|$ , with lexicographic tie-
    breaking
18: end for
19: return  $\emptyset$ 

```

3.3.1. Algorithm Pseudocode. The Pell step in Algorithm 1 means that we do not rely on a single arbitrary solution returned by a generalized Pell solver. The reference implementation enumerates all representatives returned by the underlying quadratic-form solver, keeps those satisfying $u \equiv 1 \pmod{2}$ and $v \equiv d \pmod{2}$, converts them to four-cube quadruples, and chooses the one with minimal height $H = \max_i |X_i|$. The independent verification pass described in Section 5.1 does not depend on how this solution was found.

3.3.2. Handling Obstructions. The expression $(x+1)^3 - x^3 - (y+d)^3 + y^3$ cannot be congruent to 4 modulo 9, but it can be congruent to 5. Therefore, when $n \equiv 4 \pmod{9}$ we apply the same search to $-n$ and then negate the four resulting cubes.

3.4. Why we use an indefinite Pell family. The identity above is one member of a more general two-difference family

$$(3.4) \quad (x+a)^3 - x^3 + (y+b)^3 - y^3 = n,$$

where a, b are integers. Expanding and completing squares gives an equation of the form

$$(3.5) \quad au^2 + bv^2 = m,$$

where $u = 2x + a$, $v = 2y + b$, and $m = (4n - a^3 - b^3)/3$. Positive-definite choices of a and b can produce small representations in special cases, but they do not lead to Pell-type unit orbits. We use the mixed-sign family $a = 1$, $b = -d$ because it

produces the indefinite norm equation $u^2 - dv^2 = M_{n,d}$, for which generalized Pell solvers are available.

4. HEURISTIC FOR THE SEARCH PARAMETER

4.1. Square tests for candidate parameters. The values $M_{n,d} = (4n-1+d^3)/3$ are not random integers independent of d . In particular, Lemma 3.2 shows that a prime dividing d can force the success probability for that d to be zero. We therefore separate these necessary square tests from the global representation question.

For fixed n , define $S_n(d) = 1$ if the following conditions hold:

$$d > 0, \quad d \text{ is nonsquare}, \quad d \equiv 1 - n \pmod{6},$$

and, for every prime power $p^\alpha \parallel d$, the integer $M_{n,d}$ is a square modulo p^α . Otherwise set $S_n(d) = 0$. Only the values with $S_n(d) = 1$ pass the square tests used before the Pell search.

For odd primes $p \neq 3$ with $p \nmid 4n-1$, the square test at a prime factor p of d is controlled by the quadratic character

$$\chi_n(p) = \left(\frac{(4n-1) \cdot 3^{-1}}{p} \right).$$

Thus a prime p with $\chi_n(p) = -1$ cannot divide a parameter d with $S_n(d) = 1$. For typical n , and away from the finitely many primes dividing $6(4n-1)$, these characters are modeled as taking the values ± 1 with roughly equal frequency.

4.2. Counting viable parameters. The preceding character model suggests that the prime factors of a parameter passing the square tests must lie in a set of primes of natural density about $1/2$. Standard sieve heuristics for integers supported on such a set then predict

$$\#\{d \leq t : S_n(d) = 1\} \approx C_n \frac{t}{\sqrt{\log t}},$$

where C_n depends on the exceptional behavior at small primes and at primes dividing $4n-1$. This is the point at which the dependence of $M_{n,d}$ on d enters the model: values of d failing the square tests are removed before any Bernays-type density is invoked.

4.3. Conditional Bernays-type density. Bernays' theorem for indefinite binary quadratic forms [2] motivates a global density of order $1/\sqrt{\log X}$ for integers up to X represented by a fixed indefinite form, after the necessary square tests are taken into account. We use this only as a conditional heuristic. Given $S_n(d) = 1$, model the probability that

$$u^2 - dv^2 = M_{n,d}$$

has a solution in the required parity class by

$$\Pr(\text{success at } d \mid S_n(d) = 1) \approx \frac{K_n(d)}{\sqrt{\log M_{n,d}}},$$

where $K_n(d)$ is treated as a slowly varying factor. During the early part of the search, when d^3 is small compared with n , we have $\log M_{n,d} \asymp \log n$.

The expected number of successful candidates up to t is then modeled by

$$\Lambda_n(t) \approx \sum_{\substack{d \leq t \\ S_n(d)=1}} \frac{K_n(d)}{\sqrt{\log M_{n,d}}} \approx \frac{C_n K_n t}{\sqrt{\log t} \sqrt{\log n}},$$

again up to n -dependent and slowly varying constants. Solving $\Lambda_n(t) \approx 1$ suggests that the first successful parameter should typically have size

$$t \approx \sqrt{\log n \log \log n},$$

within these constants. We do not state this as a theorem; it is a stopping-time model used to interpret the empirical distribution in Section 5.2.

5. EXPERIMENTAL VERIFICATION

5.1. Results and data availability. For each integer n , the program searches over positive nonsquare integers d as in Algorithm 1; if $n \equiv 4 \pmod{9}$, it applies the same search to $-n$ and negates the resulting four cubes. We generated a public certificate data set for every integer $0 \leq n \leq 10^9$, with each row checked by exact integer arithmetic. Solutions for negative integers $-n$ are obtained by taking the negatives of the four entries, and some small examples are listed in Table 1. The associated code and data are publicly available on Hugging Face at <https://huggingface.co/datasets/kzoacn/sum4cubes>.

5.2. Distribution analysis. Due to computational time constraints, we performed detailed statistical analysis for the reference implementation only on the range $1 \leq n \leq 10^7$. In the plots below, $d(n)$ denotes the first search parameter d for which the reference implementation finds at least one parity-compatible Pell representative. For that d , the displayed quadruple is the one of minimal height $H(n) = \max_i |x_i|$, with lexicographic tie-breaking. The figures were regenerated from this rule using bins of length 10^4 and a deterministic sample of every 5000-th integer.

Figure 1 shows sampled values of $d(n)$ on a logarithmic scale, with the largest outliers in this range highlighted. Most values of $d(n)$ are small, while the largest values are separated by several orders of magnitude.

Figure 2 gives binned quantile curves for $d(n)$. The median and high quantiles remain stable as n grows, while the bin maxima record rare large deviations.

Figure 3 shows the cumulative average of $d(n)$ together with a least-squares fit to the heuristic shape $a\sqrt{\log n \log \log n} + b$.

For the full range $1 \leq n \leq 10^9$, Appendix Tables 2 and 3 summarize the regenerated certificate data using the same rule for $d(n)$ and $H(n)$. The leading examples in these two senses are written out explicitly in Tables 4 and 5.

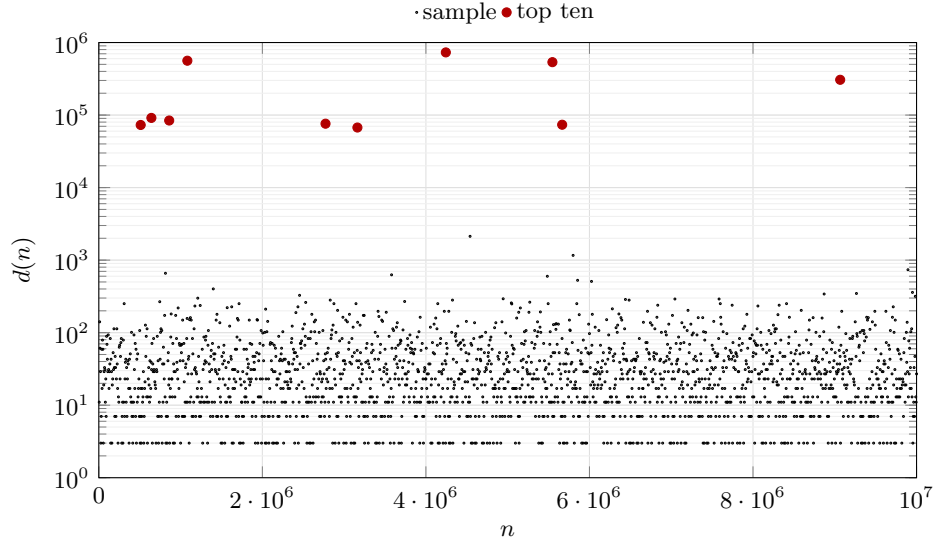


FIGURE 1. Reference search parameters $d(n)$ for $1 \leq n \leq 10^7$ on a logarithmic scale. The gray points are the deterministic sample $n \equiv 0 \pmod{5000}$, and the red points mark the ten largest observed values of $d(n)$ in this range.

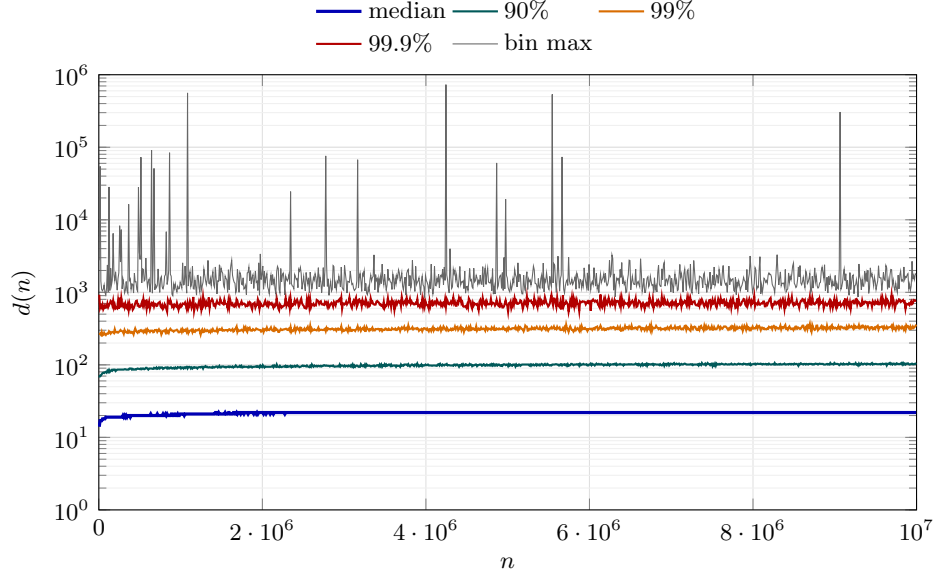


FIGURE 2. Binned quantiles of the reference search parameter $d(n)$ for $1 \leq n \leq 10^7$. Each bin contains 10^4 consecutive values of n .

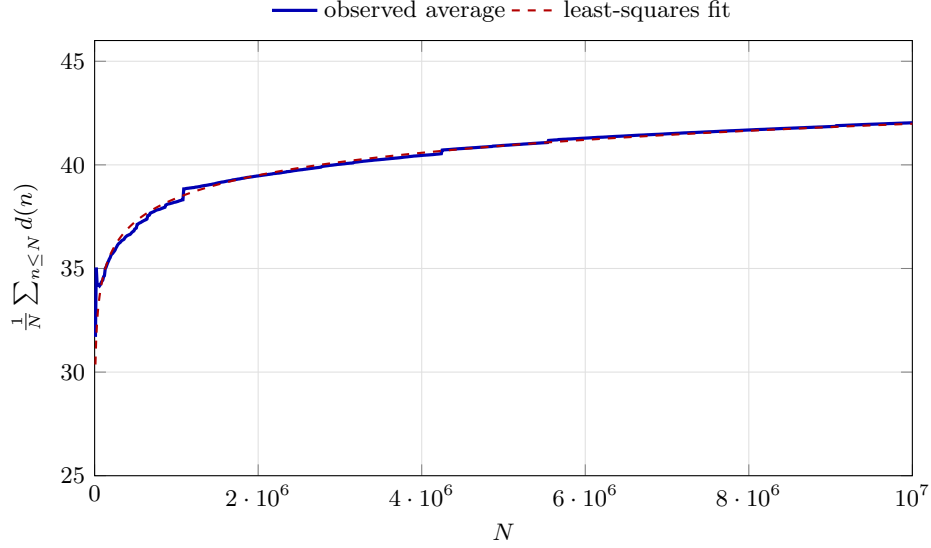


FIGURE 3. Cumulative average of the reference search parameter $d(n)$ for $1 \leq n \leq 10^7$, compared with a least-squares fit of the form $a\sqrt{\log N \log \log N} + b$.

5.3. Running Time. The algorithm also runs quickly for larger integers, e.g., $n = 9 \cdot 2^{128} + 5$:

$$\begin{aligned} n &= 3062541302288446171170371466885913903109 \\ &= 39512676350251656797^3 - 39512676350251656796^3 \\ &\quad - 1275823017616426120^3 + 1275823017616425788^3. \end{aligned}$$

This solution was found within seconds. Appendix Tables 6 and 7 give two further examples obtained from decimal prefixes of π and e , with the decimal point omitted and a prefix length chosen so that $n \equiv 4 \pmod{9}$.

5.4. Comparison with Demjanenko's Method. For $n = 254$, our algorithm yields:

$$(5.1) \quad (-12)^3 + 13^3 + 1^3 + (-6)^3 = 254,$$

while Demjanenko's method [5, 1] would classify it as a special case modulo 108, outputting four numbers with over 600 digits each (see <https://www.alpertron.com.ar/FCUBES.HTM>).

ACKNOWLEDGMENTS

We thank the anonymous reviewers for their careful reading and constructive suggestions. In particular, we are grateful for the observation that a single generalized Pell equation may have both parity-compatible and parity-incompatible solutions, which led us to state the parity conditions explicitly and to enumerate the relevant Pell representatives in the final implementation.

REFERENCES

1. Dario Alpertron, *Sum of four cubes*, <https://www.alpertron.com.ar/FCUBES.HTM>, 2020, Online calculator implementing Demjanenko's algorithms.
2. Paul Bernays, *Über die Darstellung von positiven, ganzen Zahlen durch die primitiven, binären quadratischen Formen einer nicht-quadratischen Diskriminante*, Ph.D. thesis, University of Göttingen, 1912, Often misattributed to Mathematische Annalen 72, which contains Kempner's work on pages 387-399.
3. Andrew R. Booker, *Cracking the problem with 33*, Research in Number Theory **5** (2019), no. 3, 26.
4. Andrew R. Booker and Andrew V. Sutherland, *On a question of mordell*, Proceedings of the National Academy of Sciences **118** (2021), no. 11, e2022377118.
5. V. A. Demjanenko, *On sums of four cubes*, Izvestiya Vysshikh Uchebnykh Zavedenii. Matematika **54** (1966), no. 5, 64–69, (in Russian).
6. G. H. Hardy and E. M. Wright, *An introduction to the theory of numbers*, 6th ed., Oxford University Press, Oxford, 2008, Revised by D. R. Heath-Brown and J. H. Silverman.
7. K. R. Matthews, *The diophantine equation $x^2 - dy^2 = n$, $d > 0$* , Expositiones Mathematicae **18** (2000), no. 4, 323–331.
8. J. P. Robertson, *Solving the generalized pell equation $x^2 - dy^2 = n$* , Manuscript available at <http://www.jpr2718.org/pell.pdf>, 2004.
9. Denis Simon, *Solving quadratic equations using reduced unimodular quadratic forms*, Mathematics of Computation **74** (2005), no. 251, 1531–1543.
10. The PARI Group, *Pari/gp version 2.15.5*, Univ. Bordeaux, Bordeaux, 2024, <http://pari.math.u-bordeaux.fr/>.
11. The Sage Developers, *Sagemath, the sage mathematics software system*, 2023.

APPENDIX A. SAGEMATH CODE IMPLEMENTATION

The following executable SageMath code records a reference implementation of the search logic. It was tested with SageMath 10.7. The production implementation uses the same certificate format, while the independent checker described in Section 5.1 verifies only the resulting four-cube identities. The same code is included in the source tree as `scripts/four_cubes_reference.py`.

```
from sage.all import ZZ, BinaryQF, Mod, factor, is_square
```

```
def passes_local_tests(n, d, M):
    """
    Necessary square tests used in the search.
    For each p^a || d, M must be a square modulo p^a.
    """
    for p, a in factor(d):
        q = p**a
        if not Mod(M, q).is_square():
            return False
    return True

def solve_pell_with_parity(d, M):
    """
    Yield solutions of
    u^2 - d*v^2 = M,
    u = 1 (mod 2), v = d (mod 2),
    """
```

```

    returned by Sage with these parity conditions.
    """
    Q = BinaryQF([1, 0, -d])
    for u, v in Q.solve_integer(M, _flag=3):
        u, v = ZZ(u), ZZ(v)
        if (u - 1) % 2 == 0 and (v - d) % 2 == 0:
            yield u, v

def quadruple_from_pell(d, u, v):
    x = (u - 1) // 2
    y = (v - d) // 2
    return (x + 1, -x, -(y + d), y)

def height(xs):
    return max(abs(x) for x in xs)

def solve(n, LIMIT=10**8, return_d=False):
    """
    Solve  $x_1^3 + x_2^3 + x_3^3 + x_4^3 = n$  by the Pell family.
    """
    n = ZZ(n)
    if n % 9 == 4:
        sol = solve(-n, LIMIT, return_d=True)
        if sol is None:
            return None
        x1, x2, x3, x4, d = sol
        out = (-x1, -x2, -x3, -x4)
        return out + (d,) if return_d else out

    residue = ZZ((1 - n) % 6)
    if residue == 0:
        residue = ZZ(6)

    for d in range(residue, LIMIT + 1, 6):
        d = ZZ(d)
        if is_square(d):
            continue

        numerator = 4*n - 1 + d**3
        if numerator % 3 != 0:
            continue
        M = numerator // 3

        if not passes_local_tests(n, d, M):
            continue

```

```

candidates = [
    quadruple_from_pell(d, u, v)
    for u, v in solve_pell_with_parity(d, M)
]
if not candidates:
    continue

out = min(
    candidates,
    key=lambda xs: (height(xs), tuple(xs)),
)
return out + (d,) if return_d else out

return None

def verify(n, xs):
    return sum(ZZ(x)**3 for x in xs) == ZZ(n)

for n in [254, 314159265358979323, 27182818284590452]:
    sol = solve(n, LIMIT=2000, return_d=True)
    assert sol is not None
    assert verify(n, sol[:4])
    print(n, sol)

```

APPENDIX B. TABLES

For a representation produced by the reference implementation in Algorithm 1, we write $d(n)$ for the first search parameter found by the algorithm and

$$H(n) = \max_{1 \leq i \leq 4} |x_i|$$

for the height of the corresponding minimal-height representative at that search parameter. The first table below lists small examples explicitly. The following two tables select outlying instances from the regenerated certificate data for the full verification range $1 \leq n \leq 10^9$. The column $\ell(H(n))$ denotes the number of decimal digits of $H(n)$.

Table 1: Sum of four cubes representations for $1 \leq n \leq 100$

n	$x_1^3 + x_2^3 + x_3^3 + x_4^3$	d
1	$(-6)^3 + 7^3 + (-1)^3 + (-5)^3$	6
2	$(-3)^3 + 4^3 + (-3)^3 + (-2)^3$	5
3	$235^3 + (-234)^3 + (-56)^3 + 22^3$	34
4	$4^3 + (-5)^3 + 4^3 + 1^3$	5
5	$2^3 + (-1)^3 + (-1)^3 + (-1)^3$	2
6	$(-58)^3 + 59^3 + (-21)^3 + (-10)^3$	31

Continued on next page

n	$x_1^3 + x_2^3 + x_3^3 + x_4^3$	d
7	$5^3 + (-4)^3 + (-3)^3 + (-3)^3$	6
8	$14^3 + (-13)^3 + (-8)^3 + (-3)^3$	11
9	$(-108)^3 + 109^3 + (-27)^3 + (-25)^3$	52
10	$(-2)^3 + 3^3 + (-2)^3 + (-1)^3$	3
11	$(-2)^3 + 3^3 + (-2)^3 + 0^3$	2
12	$(-186)^3 + 187^3 + (-47)^3 + (-8)^3$	55
13	$10^3 + (-11)^3 + 7^3 + 1^3$	8
14	$(-235)^3 + 236^3 + 2^3 + (-55)^3$	53
15	$842^3 + (-841)^3 + (-132)^3 + 56^3$	76
16	$(-19)^3 + 20^3 + (-10)^3 + (-5)^3$	15
17	$3^3 + (-2)^3 + (-1)^3 + (-1)^3$	2
18	$(-95)^3 + 96^3 + (-7)^3 + (-30)^3$	37
19	$(-5)^3 + 6^3 + (-4)^3 + (-2)^3$	6
20	$(-20)^3 + 21^3 + (-9)^3 + (-8)^3$	17
21	$10^3 + (-9)^3 + (-5)^3 + (-5)^3$	10
22	$85^3 + (-86)^3 + 28^3 + 1^3$	29
23	$146^3 + (-145)^3 + (-40)^3 + 8^3$	32
24	$(-16)^3 + 17^3 + (-9)^3 + (-4)^3$	13
25	$(-136)^3 + 137^3 + (-38)^3 + (-10)^3$	48
26	$(-4)^3 + 5^3 + (-3)^3 + (-2)^3$	5
27	$(-11)^3 + 12^3 + (-7)^3 + (-3)^3$	10
28	$(-3)^3 + 4^3 + (-2)^3 + (-1)^3$	3
29	$(-3)^3 + 4^3 + (-2)^3 + 0^3$	2
30	$120^3 + (-119)^3 + (-35)^3 + 4^3$	31
31	$229687^3 + (-229688)^3 + 7390^3 + (-6260)^3$	1130
32	$431^3 + (-430)^3 + (-94)^3 + 65^3$	29
33	$59^3 + (-58)^3 + (-19)^3 + (-15)^3$	34
34	$(-4)^3 + 5^3 + (-3)^3 + 0^3$	3
35	$4^3 + (-3)^3 + (-1)^3 + (-1)^3$	2
36	$(-6)^3 + 7^3 + (-4)^3 + (-3)^3$	7
37	$6^3 + (-5)^3 + (-3)^3 + (-3)^3$	6
38	$(-41)^3 + 42^3 + (-17)^3 + (-6)^3$	23
39	$117^3 + (-116)^3 + (-35)^3 + 13^3$	22
40	$148^3 + (-149)^3 + 40^3 + 13^3$	53
41	$8^3 + (-7)^3 + (-4)^3 + (-4)^3$	8
42	$(-109)^3 + 110^3 + 15^3 + (-34)^3$	19
43	$(-7)^3 + 8^3 + (-5)^3 + (-1)^3$	6
44	$(-7)^3 + 8^3 + (-5)^3 + 0^3$	5
45	$(-44)^3 + 45^3 + (-18)^3 + (-4)^3$	22
46	$25^3 + (-24)^3 + (-3)^3 + (-12)^3$	15
47	$(-9)^3 + 10^3 + (-6)^3 + (-2)^3$	8
48	$(-21)^3 + 22^3 + (-11)^3 + (-2)^3$	13
49	$(-2)^3 + 1^3 + 4^3 + (-2)^3$	2
50	$(-76)^3 + 77^3 + (-22)^3 + (-19)^3$	41
51	$(-10)^3 + 11^3 + (-6)^3 + (-4)^3$	10
52	$(-4)^3 + 5^3 + (-2)^3 + (-1)^3$	3

Continued on next page

n	$x_1^3 + x_2^3 + x_3^3 + x_4^3$	d
53	$(-4)^3 + 5^3 + (-2)^3 + 0^3$	2
54	$(-9)^3 + 10^3 + (-6)^3 + (-1)^3$	7
55	$(-6)^3 + 7^3 + (-4)^3 + (-2)^3$	6
56	$(-5)^3 + 6^3 + (-3)^3 + (-2)^3$	5
57	$(-21)^3 + 22^3 + 1^3 + (-11)^3$	10
58	$1^3 + (-2)^3 + 4^3 + 1^3$	5
59	$5^3 + (-4)^3 + (-1)^3 + (-1)^3$	2
60	$1202^3 + (-1201)^3 + (-163)^3 + 0^3$	163
61	$(-16)^3 + 17^3 + (-9)^3 + (-3)^3$	12
62	$7^3 + (-6)^3 + (-4)^3 + (-1)^3$	5
63	$(-30)^3 + 31^3 + (-12)^3 + (-10)^3$	22
64	$(-5)^3 + 6^3 + (-3)^3 + 0^3$	3
65	$(-5)^3 + 6^3 + (-3)^3 + 1^3$	2
66	$(-198)^3 + 199^3 + (-50)^3 + 19^3$	31
67	$(-5)^3 + 4^3 + 4^3 + 4^3$	8
68	$257^3 + (-256)^3 + (-58)^3 + (-13)^3$	71
69	$(-148)^3 + 149^3 + (-42)^3 + 20^3$	22
70	$(-58)^3 + 59^3 + (-20)^3 + (-13)^3$	33
71	$(-6)^3 + 7^3 + 2^3 + (-4)^3$	2
72	$(-14)^3 + 15^3 + (-7)^3 + (-6)^3$	13
73	$7^3 + (-6)^3 + (-3)^3 + (-3)^3$	6
74	$25^3 + (-24)^3 + (-12)^3 + 1^3$	11
75	$(-53)^3 + 54^3 + (-20)^3 + (-8)^3$	28
76	$10^3 + (-11)^3 + 7^3 + 4^3$	11
77	$(-19)^3 + 20^3 + (-10)^3 + (-4)^3$	14
78	$(-7)^3 + 8^3 + (-4)^3 + (-3)^3$	7
79	$(-13)^3 + 14^3 + (-7)^3 + (-5)^3$	12
80	$(-16)^3 + 17^3 + (-9)^3 + (-2)^3$	11
81	$11^3 + (-10)^3 + (-5)^3 + (-5)^3$	10
82	$(-5)^3 + 6^3 + (-2)^3 + (-1)^3$	3
83	$(-5)^3 + 6^3 + (-2)^3 + 0^3$	2
84	$(-8)^3 + 9^3 + (-5)^3 + (-2)^3$	7
85	$(-86)^3 + 85^3 + 28^3 + 4^3$	32
86	$(-106795)^3 + 106796^3 + (-4039)^3 + 3164^3$	875
87	$(-16)^3 + 17^3 + (-1)^3 + (-9)^3$	10
88	$473^3 + (-472)^3 + (-90)^3 + 39^3$	51
89	$6^3 + (-5)^3 + (-1)^3 + (-1)^3$	2
90	$(-188)^3 + 189^3 + (-43)^3 + (-30)^3$	73
91	$9^3 + (-8)^3 + (-5)^3 + (-1)^3$	6
92	$(-6)^3 + 7^3 + (-3)^3 + (-2)^3$	5
93	$165^3 + (-164)^3 + (-44)^3 + 16^3$	28
94	$535^3 + (-536)^3 + 94^3 + 31^3$	125
95	$(-130)^3 + 131^3 + (-7)^3 + (-37)^3$	44
96	$17^3 + (-16)^3 + (-9)^3 + 2^3$	7
97	$(-7)^3 + 8^3 + (-4)^3 + (-2)^3$	6
98	$26^3 + (-25)^3 + (-12)^3 + (-5)^3$	17

Continued on next page

n	$x_1^3 + x_2^3 + x_3^3 + x_4^3$	d
99	$13^3 + (-12)^3 + (-7)^3 + (-3)^3$	10
100	$7^3 + (-6)^3 + (-3)^3 + 0^3$	3

TABLE 2. Largest recorded values of $d(n)$ for $1 \leq n \leq 10^9$

Rank	n	d	$\ell(H)$	$\log_{10}(H/n)$
1	936384844	5508491	66	56.516
2	630252220	5170439	17	7.476
3	172116347	3042818	26	17.569
4	720368492	2483135	23	14.040
5	62809898	2259611	18	9.591
6	424484014	1890905	41	31.940
7	984206885	1774796	27	17.274
8	233143708	1759577	16	6.785
9	200441272	1539353	33	24.389
10	699315232	1421681	14	4.933

TABLE 3. Largest recorded values of $H(n)$ for $1 \leq n \leq 10^9$

Rank	n	d	$\ell(H)$	$\log_{10}(H/n)$
1	340984507	1248734	131	122.036
2	936384844	5508491	66	56.516
3	139360442	473225	58	49.605
4	812091910	1387697	57	48.084
5	298597495	908216	51	41.866
6	243310568	676853	47	38.513
7	28875326	218663	44	36.337
8	424484014	1890905	41	31.940
9	584188196	94217	35	25.611
10	200441272	1539353	33	24.389

B.1. Explicit representations for two outliers. The following two tables give the complete representations for the leading outliers in Tables 2 and 3. In these tables, spaces in decimal expansions are only digit separators, inserted in blocks of four for readability.

TABLE 4. Explicit certificate representation for $n = 936384844$, the largest recorded value of $d(n)$

Variable	Value
x_1	30 7230 6200 3619 8887 6003 3192 9948 7199 7824 7989 0641 1712 7246 2307 8629 5941
x_2	-30 7230 6200 3619 8887 6003 3192 9948 7199 7824 7989 0641 1712 7246 2307 8629 5942
x_3	130 9025 7064 0549 0465 2247 7828 1922 0428 4025 4310 9694 6153 1423 2834 8471
x_4	-130 9025 7064 0549 0465 2247 7828 1922 0428 4025 4310 9694 6153 1423 2283 9980

$$936384844 = x_1^3 + x_2^3 + x_3^3 + x_4^3,$$

$$d(936384844) = 5508491, \quad H(936384844) = |x_2|.$$

TABLE 5. Explicit certificate representation for $n = 340984507$, the largest recorded value of $H(n)$

Variable	Value
x_1	370 5739 4133 4654 4780 1216 2360 0386 9785 7350 5408 0417 7220 5119 4147 2288 7378 2670 5430 4572 5176 0745 4543 2018 9726 4367 8218 0837 8821 0789 8314 1962 2346
x_2	-370 5739 4133 4654 4780 1216 2360 0386 9785 7350 5408 0417 7220 5119 4147 2288 7378 2670 5430 4572 5176 0745 4543 2018 9726 4367 8218 0837 8821 0789 8314 1962 2347
x_3	-3316 1938 4003 7246 7141 7215 0547 4292 0031 4035 9034 0369 9705 9982 1979 2976 7723 6077 8586 5629 2076 3931 3899 4614 3197 6791 5676 7171 1428 1184 4392 6711
x_4	3316 1938 4003 7246 7141 7215 0547 4292 0031 4035 9034 0369 9705 9982 1979 2976 7723 6077 8586 5629 2076 3931 3899 4614 3197 6791 5676 7171 1428 1184 4517 5445

$$340984507 = x_1^3 + x_2^3 + x_3^3 + x_4^3,$$

$$d(340984507) = 1248734, \quad H(340984507) = |x_2|.$$

B.2. Decimal-prefix examples. The following two tables give additional examples obtained from decimal prefixes of π and e . The digit count includes the digit before the decimal point; the decimal point is then omitted. Both selected prefixes satisfy $n \equiv 4 \pmod{9}$. The explicit representations are shown in Tables 6 and 7.

TABLE 6. Explicit representation for the 18-digit decimal prefix of π

Variable	Value
x_1	16411235899
x_2	-16411235900
x_3	525952624
x_4	-525951650

$$314159265358979323 = x_1^3 + x_2^3 + x_3^3 + x_4^3,$$

$$d(314159265358979323) = 974, \quad H(314159265358979323) = |x_2|.$$

TABLE 7. Explicit representation for the 17-digit decimal prefix of e

Variable	Value
x_1	6482848
x_2	-6482849
x_3	-19894238
x_4	19894261

$$27182818284590452 = x_1^3 + x_2^3 + x_3^3 + x_4^3,$$

$$d(27182818284590452) = 23, \quad H(27182818284590452) = |x_4|.$$

INSTITUTE FOR ADVANCED STUDY, TSINGHUA UNIVERSITY, BEIJING, CHINA
Email address: kaiyizhang@tsinghua.edu.cn

SCHOOL OF COMPUTER SCIENCE, SHANGHAI JIAO TONG UNIVERSITY, SHANGHAI, CHINA
Email address: shikaku@sjtu.edu.cn

INSTITUTE FOR ADVANCED STUDY, TSINGHUA UNIVERSITY, BEIJING, CHINA; STATE KEY LABORATORY OF CRYPTOGRAPHY AND DIGITAL ECONOMY SECURITY, TSINGHUA UNIVERSITY, BEIJING, CHINA; SCHOOL OF CRYPTOLOGIC SCIENCE AND ENGINEERING, SHANDONG UNIVERSITY, JINAN, CHINA; ZHONGGUANCUN LABORATORY, BEIJING, CHINA; NATIONAL FINANCIAL CRYPTOGRAPHY RESEARCH CENTER, BEIJING, CHINA

Email address: anyuwang@tsinghua.edu.cn

INSTITUTE FOR ADVANCED STUDY, TSINGHUA UNIVERSITY, BEIJING, CHINA; STATE KEY LABORATORY OF CRYPTOGRAPHY AND DIGITAL ECONOMY SECURITY, TSINGHUA UNIVERSITY, BEIJING, CHINA; SCHOOL OF CRYPTOLOGIC SCIENCE AND ENGINEERING, SHANDONG UNIVERSITY, JINAN, CHINA; ZHONGGUANCUN LABORATORY, BEIJING, CHINA; NATIONAL FINANCIAL CRYPTOGRAPHY RESEARCH CENTER, BEIJING, CHINA; SHANDONG INSTITUTE OF BLOCKCHAIN, SHANDONG, CHINA

Email address: xiaoyunwang@tsinghua.edu.cn